지능형 DB 11주

DDL

SQL 개요

데이터베이스에 저장된 데이터를 추출 또는 입력, 수정, 삭제하는 스크립트 언어

DML (Data Manipulation Language) – 주로 개발자가 사용

SELECT, INSERT, UPDATE, DELETE

DDL (Data Definition Language) – 주로 DBA(데이터베이스관리자)가 사용

CREATE, ALTER, DROP

DCL (Data Control Language) - 주로 DBA(데이터베이스관리자)가 사용

GRANT, REVOKE

데이터 저장 및 취소

공식적 영구히 반영하는 명령어

COMMIT : 모든 변경사항(입력, 수정, 삭제)을 반영하는 명령어

ROLLBACK : 모든 변경사항(입력, 수정, 삭제)을 취소하는 명령어

ROLLBACK은 직전 COMMIT 이후의 모든 변경사항을 취소한다.

데이터 저장 및 취소

사원테이블(emp)에 새로운 사원의 정보를 입력하고 공식적으로 반영하시오.

empno : 1122

ename : 'JACK'

sal = 3000

deptno = 20

데이터 저장 및 취소

INSERT into emp(empno, ename, sal, deptno)

VALUES (1122, 'JACK', 3000, 20);

COMMIT;

/* 입력된 데이터취소는 불가능하며 일반적으로 DELETE명령문을 통해 삭제가 가능하다.*/

/* 다만 금융결제일 경우, 만일 CU에서 물건을 구매한 후 카드 결제 후 잘못된 계산을 인지하여 취소를 해야 된다면 잘못된 액수의 마이너스 값으로 입력을 다시 하고(취소전표) 이어서 새로운 가격으로 입력하게 된다. */

데이터 저장 및 취소

사원 테이블 (emp)에 다음의 정보를 이용하여 UPDATE문을 작성하고 취소하시오.

사원명 : ‘SCOTT’

급여 : 4000

데이터 저장 및 취소

```
UPDATE emp
SET sal = 4000
WHERE ename = 'SCOTT';
ROLLBACK;
/* ROLLBACK의 범위는 직전 COMMIT 직후 ~ ROLLBACK을 만난 범위까지 취소된다. */
```

일반 테이블 생성하기

사원번호, 이름, 급여, 입사일을 저장하는 테이블 'emp1'을 만들어보자.

형식

```
CREATE TABLE emp1
( empno number(10),
  ename varchar2(10),
  sal number(10,2),
  hiredate date,
  deptno number(10));
```

일반 테이블 생성하기

테이블 명 규칙

반드시 문자로 시작

최대 30자(영문자 기준)

대소문자, 숫자 포함

특수문자는 $, _ , # 만 가능

NUMBER(10,2)는 소수점 포함 최대 10자까지 이며 소수점 이하 2자리를 지정하며 따라서 정수부문은 최대 8자 까지 가능

DATE 형은 기원전 4712년 1월 1일 부터 9999년 12월 31일 까지 저장 가능

서브 쿼리를 이용한 데이터 입력

emp1에 부서번호가 10인 사원들의 사원번호, 사원명, 급여, 부서번호를 서브 쿼리를 이용하여 작성하시오.

언제 이런 쿼리를 사용할까? (임시로 커스트마이징한 테이블을 테스트용으로 사용)

```
INSERT INTO emp1 (empno, ename, sal, deptno)
/* VALUES */
  SELECT empno, ename, sal, deptno
      FROM emp
      WHERE deptno = 10; /* 3건 입력 완료 */
```

서브 쿼리를 이용한 데이터 수정

직업이 'CLERK'인 사원들의 급여를 'ALLEN'의 급여로 변경해보자.

```
UPDATE emp
SET sal = ( select
            from
            where                    )
WHERE job = 'CLERK'; /* 4건이 수정되었습니다 */
```

서브 쿼리를 이용한 데이터 삭제

'SCOTT'보다 급여를 많이 받는 사원들을 삭제해보자.

```
DELETE FROM emp
WHERE sal > ( select
              from
              where          ); /* 1행이 제거되었습니다. */
```

서브 쿼리를 이용한 데이터 삭제

보충문제 2번을 작성해보자.

SELF – SUB QUERY를 작성해야 풀어낼 수 있는 문제임.

임시 테이블 생성 (테스트 목적)

```
CREATE GLOBAL TEMPORARY TABLE emp2
( empno number(10),
  ename varchar2(10),
  sal number(10))
  ON COMMIT DELETE ROWS;
/* 임시테이블 생성 후 데이터 입력하고 만일 COMMIT을 만나면 데이터 삭제*/
```

임시 테이블 생성 (테스트 목적)

```
INSERT into emp2 values (1111, 'scott', 1000);
INSERT into emp2 values (2222, 'jack', 2000);
COMMIT;

SELECT * FROM emp2;
```

임시 테이블 생성 (테스트 목적)

```
CREATE GLOBAL TEMPORARY TABLE emp2
( empno number(10),
  ename number(10),
  sal number(10))
  ON COMMIT PRESERVE ROWS;
/* 임시테이블 생성 후 데이터 입력하고 만일 COMMIT을 만나면 데이터 보관*/
/* 단, 로그아웃 하면 데이터 사라짐!! 그러면 테이블도 사라질까? */
```

임시 테이블 생성 (테스트 목적)

```
INSERT into emp2 values (1111, 'scott', 1000);
INSERT into emp2 values (2222, 'jack', 2000);
COMMIT;

SELECT * FROM emp2;
```

VIEW (커스트마이징 테이블)

가상 테이블이다.

이전에 배운 임시 테이블과 달리 '공식적'인 테이블이다.

몇 개의 뷰 테이블을 어떠한 구조로 만들지는 철저히 노하우이다.

불필요하게 많이 만들면 입력/갱신/삭제 시 성능하락이 있을 수 있다.

원본 테이블이 안전하게 보관되기 때문에 보안성이 좋다.

업계에서는 반드시 만들게 되는 가상 테이블이다. (임시 테이블이 아님에 주의)

과도한 조인을 회피할 수 있다.

보통 테이블명을 'view_테이블명' 또는 'v_테이블명'으로 작성한다.

VIEW (커스트마이징 테이블)

형식

CREATE VIEW 뷰테이블명

AS

SELECT

FROM

WHERE ;

VIEW (커스트마이징 테이블)

직업이 'MANAGER'인 사원들의 사원번호, 사원명, 급여, 직업, 부서번호로 구성하는 'view_emp' 뷰 테이블을 작성해 보자.

```
CREATE VIEW view_emp
AS
SELECT empno, ename, sal, job, deptno
FROM emp
WHERE  job = 'MANAGER';
```

VIEW (커스트마이징 테이블)

직업이 'MANAGER'인 사원들의 사원번호, 사원명, 급여, 직업, 부서명으로 구성하는 'view_emp' 뷰 테이블을 작성해 보자.

```
CREATE VIEW view_manager
AS
SELECT
FROM
WHERE ;
```

복합 VIEW

뷰에 그룹 함수가 포함될 때 복합뷰라고 칭한다.

업계에서는 따로 구분하지 않고 뷰라는 통칭으로 불린다.

	단순 view	**복합 view**
테이블 수	1	2개 이상(업계 대부분)
그룹함수	포함 안함	포함
원본 테이블 갱신 여부	갱신가능*	갱신 불가능

업계에서는 view에 삽입/갱신/삭제 행위를 사실상 하지 않는다.

복합 VIEW

부서번호와 부서번호별 평균 급여를 출력하는 뷰를 생성해보자.

```
CREATE VIEW view_emp2
AS
SELECT deptno, round(avg(sal)) avg_sal
FROM emp
GROUP by deptno;
```

복합 VIEW

보충문제 2번을 작성해보자.

앞장의 view를 이용해보자.

만일 view을 사용하지 않고 작성하면 어떻게 작성될 수 있을까?

복합 VIEW

```
SELECT e.ename, e.sal, e.deptno, v.avg_sal
FROM emp e, (select deptno, reound(avg(sal)) avg_sal
                from emp
                group by deptno) v
WHERE e.deptno = v.deptno and e.sal > v.avg_sal;
```

INDEX (검색 성능 향상)

인덱스는 업계에서 반드시 생성하는 오브젝트이다.

테이블에서 조건에 맞추어 내부적으로 부분 정렬한다.

어느 테이블에 어떠한 조건으로 몇 개의 인덱스를 구성하는지는 노하우이다.

불필요하게 많은 인덱스를 구성하면 오히려 검색속도를 저해할 수 있다.

입력/수정/삭제 시 re-index를 해야 하기 때문에 잦은 입력/수정/삭제가 발생하는환경에서는 인덱스 생성에 유의해야 한다.

INDEX (검색 성능 향상)

형식

CREATE INDEX 인덱스명

ON 테이블명(어트리뷰트명);

Emp테이블에 급여를 기준으로 인덱스를 생성해보자.

CREATE INDEX idx_emp

ON emp(sal);

SEQUENCE 자동 순번 생성하기

입력이 수행될 때 마다 순번을 '1'씩 증가시키며 생성시키고 싶다면?

1~100번 까지 출력하는 시퀀스를 생성해 보자.

```
CREATE SEQUENCE SEQ1
START WITH 1
INCREMENT BY 1
MAXVALUE 100
NOCYCLE;  /* 100번 생성후 다시 1번 부터 생성하고 싶으면 'CYCLE' */
```

SEQUENCE 자동 순번 생성하기

실습을 위해 하나의 테이블을 생성해보자.

```
CREATE TABLE SEQ_TEMP
EMPNO NUMBER(10)
ENAME VARCHAR2(10),
SAL NUMBER(10));
```

SEQUENCE 자동 순번 생성하기

```
INSERT INTO SEQ_TEMP VALUES (SEQ1.NEXTVAL, 'FIRST', 1000);
INSERT INTO SEQ_TEMP VALUES (SEQ1.NEXTVAL, 'SECOND', 2000);
SELECT * FROM SEQ_TEMP;
```

장점 : 사번이 중복되어 입력되지 않는다.(사전적 예방, 무결성 보장)

단점 : 사번의 경우 회사정책상 다양한 조합으로 구성할 수 있어 SEQUECE 생성에 있어 제한이 많이 따른다.

주요키를 통한 데이터 무결성 높이기

테이블 생성시 주요키를 설정함으로써 무결성을 높일 수 있다.

무결성 조건 : PK설정 시 'NOT NULL'이 자동 설정되며 데이터 누락이 안되어 데이터 품질을 높일 수 있다.

PK설정 시 동일 테이블에서 중복값이 허용이 안됨.

PK 이외의 컬럼도 NOT NULL을 부여할 수 있음.

회원가입시 이름의 경우 중요한 사항이어서 PK가 아니어도 보통 NOT NULL 부여

주요키를 통한 데이터 무결성 높이기

```
CREATE TABLE DEPT_TEMP
( DEPTNO NUMBER(10) PRIMARY KEY,
  DNAME VARCHAR2(10),
  LOC VARCHAR2(10));
```

PK가 없어도 테이블은 생성되지만 업계에선 절대 금지 사항입니다.

데이터모델링 당시 PK가 설정되어 진행됩니다.

UNIQUE

테이블의 각 컬럼에는 중복데이터를 방지 하기 위해 'UNIQUE'를 첨가하여 테이블을 작성할 수 있습니다.

PK와는 다르게 모든 컬럼에 적용할 수 있고 NULL값은 허용됩니다.

```
CREATE TABLE DEPT_UNIQUE
( DEPTNO NUMBER(10) PRIMARY KEY,
  DNAME VARCHAR2(10),
  LOC VARCHAR2(10),
  UNIQUE(LOC));
```

NOT NULL

```
CREATE TABLE DEPT_NULL
( DEPTNO NUMBER(10) PRIMARY KEY,
  DNAME VARCHAR2(10) not null,
  LOC VARCHAR2(10),
  UNIQUE(LOC));
```

CHECK

에를 들어 사원 테이블을 생성하는데 급여가 0~9000까지만 허용되게 생성하고 싶다면...

```
CREATE TABLE CHECK_TEMP

EMPNO NUMBER(100)

ENAME VARCHAR2(10),

SAL NUMBER(10) CHECK ( SAL >= 0));
```